In [ ]:
```
import pandas as pd
dataset=pd.read_csv("insurance.csv")
dataset
```

In [ ]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

In [ ]:
```
x=dataset[['age', 'bmi', 'children','sex_male', 'smoker_yes','region_northwest','region_southeast','region_southwest']]
y=dataset['charges']
print(x)
print(y)
```

In [ ]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.20,random_state=42)
y_test
```

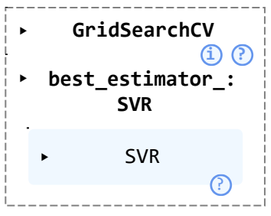
In [5]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)
```

In [8]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVR
param_grid={'kernel':['rbf','poly','sigmoid','linear'],
            'C':[10,100,1000,2000,3000],
            'gamma':['auto','scale']}

grid=GridSearchCV(SVR(),param_grid,refit=True,verbose=3,n_jobs=-1)
grid.fit(x_train,y_train)
```

```
Fitting 5 folds for each of 40 candidates, totalling 200 fits
```

Out[8]:



In [12]:
```
re=grid.cv_results_
print("The R_scorevalueforbestparameter{}:".format(grid.best_params_))
```

```
The R_scorevalueforbestparameter{'C': 3000, 'gamma': 'auto', 'kernel': 'poly'}:
```

In [13]:
```
table=pd.DataFrame.from_dict(re)
table
```

Out[13]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C	param_gamma	param_kernel	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_s
0	0.059297	0.003078	0.030333	0.002233	10	auto	rbf	{'C': 10, 'gamma': 'auto', 'kernel': 'rbf'}	-0.083704	-0.137685	-0.041203	-0.073415	-0.024180	-0.072037	0.039214	
1	0.039940	0.003146	0.010615	0.006208	10	auto	poly	{'C': 10, 'gamma': 'auto', 'kernel': 'poly'}	-0.066668	-0.120308	-0.023292	-0.059093	-0.010000	-0.055872	0.038587	
2	0.064179	0.006904	0.011955	0.000974	10	auto	sigmoid	{'C': 10, 'gamma': 'auto', 'kernel': 'sigmoid'}	-0.040776	-0.089019	0.009181	-0.033218	0.022846	-0.026197	0.039662	
3	0.041648	0.005495	0.007338	0.000559	10	auto	linear	{'C': 10, 'gamma': 'auto', 'kernel': 'linear'}	0.375975	0.402596	0.450048	0.355504	0.439389	0.404702	0.036082	
4	0.050006	0.002152	0.027751	0.001775	10	scale	rbf	{'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}	-0.083770	-0.137723	-0.041212	-0.073359	-0.024136	-0.072040	0.039239	
5	0.037319	0.000342	0.007676	0.000147	10	scale	poly	{'C': 10, 'gamma': 'scale', 'kernel': 'poly'}	-0.065983	-0.119973	-0.023198	-0.059698	-0.010351	-0.055841	0.038381	
6	0.052988	0.000916	0.010734	0.000275	10	scale	sigmoid	{'C': 10, 'gamma': 'scale', 'kernel': 'sigmoid'}	-0.040537	-0.088862	0.009220	-0.033470	0.022681	-0.026194	0.039570	
7	0.039650	0.002243	0.006952	0.000234	10	scale	linear	{'C': 10, 'gamma': 'scale', 'kernel': 'linear'}	0.375975	0.402596	0.450048	0.355504	0.439389	0.404702	0.036082	
8	0.050184	0.001334	0.025999	0.000512	100	auto	rbf	{'C': 100, 'gamma': 'auto', 'kernel': 'rbf'}	0.119717	0.098300	0.194795	0.133817	0.207817	0.150889	0.042889	
9	0.039178	0.000843	0.007549	0.000261	100	auto	poly	{'C': 100, 'gamma': 'auto', 'kernel': 'poly'}	0.279198	0.234564	0.330862	0.258190	0.305122	0.281587	0.033900	
10	0.054822	0.002181	0.010939	0.000272	100	auto	sigmoid	{'C': 100, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.396719	0.428268	0.464029	0.374083	0.460546	0.424729	0.035183	
11	0.049931	0.006891	0.006956	0.000152	100	auto	linear	{'C': 100, 'gamma': 'auto', 'kernel': 'linear'}	0.560642	0.638680	0.617470	0.560400	0.611113	0.597661	0.031669	
12	0.051487	0.001303	0.026371	0.000485	100	scale	rbf	{'C': 100, 'gamma': 'scale', 'kernel': 'rbf'}	0.119214	0.097982	0.194721	0.134236	0.208167	0.150864	0.043086	
13	0.038932	0.000538	0.007488	0.000322	100	scale	poly	{'C': 100, 'gamma': 'scale', 'kernel': 'poly'}	0.283121	0.236939	0.331394	0.254560	0.302661	0.281735	0.033618	
14	0.054138	0.001604	0.011900	0.001682	100	scale	sigmoid	{'C': 100, 'gamma': 'scale', 'kernel': 'sigmoid'}	0.394481	0.429075	0.463869	0.375535	0.460347	0.424661	0.035084	
15	0.053991	0.007341	0.008031	0.000935	100	scale	linear	{'C': 100, 'gamma': 'scale', 'kernel': 'linear'}	0.560642	0.638680	0.617470	0.560400	0.611113	0.597661	0.031669	
16	0.072111	0.003762	0.029003	0.001102	1000	auto	rbf	{'C': 1000, 'gamma': 'auto', 'kernel': 'rbf'}	0.651045	0.708112	0.673643	0.625572	0.688915	0.669457	0.028828	
17	0.066305	0.005543	0.009264	0.001349	1000	auto	poly	{'C': 1000, 'gamma': 'auto', 'kernel': 'poly'}	0.758451	0.783164	0.735832	0.667415	0.743247	0.737622	0.038657	
18	0.071628	0.003043	0.012536	0.001712	1000	auto	sigmoid	{'C': 1000, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.437972	0.586161	0.529324	0.503912	0.540355	0.519545	0.048713	
19	0.077389	0.011959	0.008302	0.000899	1000	auto	linear	{'C': 1000, 'gamma': 'auto', 'kernel': 'linear'}	0.692944	0.714152	0.651348	0.519884	0.716166	0.658899	0.073313	

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C	param_gamma	param_kernel	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_s
20	0.072225	0.003075	0.029520	0.001958	1000	scale	rbf	{'C': 1000, 'gamma': 'scale', 'kernel': 'rbf'}	0.650772	0.707907	0.673591	0.625845	0.689144	0.669452	0.028755	
21	0.069920	0.009317	0.008329	0.001270	1000	scale	poly	{'C': 1000, 'gamma': 'scale', 'kernel': 'poly'}	0.759182	0.784079	0.736059	0.666390	0.742594	0.737661	0.039304	
22	0.071724	0.004647	0.012109	0.000833	1000	scale	sigmoid	{'C': 1000, 'gamma': 'scale', 'kernel': 'sigmo...	0.430708	0.610326	0.550406	0.515166	0.536050	0.528531	0.058264	
23	0.073340	0.011386	0.008277	0.000421	1000	scale	linear	{'C': 1000, 'gamma': 'scale', 'kernel': 'linear'}	0.692944	0.714152	0.651348	0.519884	0.716166	0.658899	0.073313	
24	0.089934	0.007133	0.028683	0.001528	2000	auto	rbf	{'C': 2000, 'gamma': 'auto', 'kernel': 'rbf'}	0.771999	0.818061	0.759579	0.705479	0.798487	0.770721	0.038456	
25	0.098165	0.022927	0.008509	0.000796	2000	auto	poly	{'C': 2000, 'gamma': 'auto', 'kernel': 'poly'}	0.795661	0.851134	0.773266	0.693558	0.792488	0.781221	0.050934	
26	0.066349	0.001646	0.012196	0.000592	2000	auto	sigmoid	{'C': 2000, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.251201	0.500540	0.428070	0.326275	0.371733	0.375564	0.085189	
27	0.194094	0.235720	0.006941	0.000170	2000	auto	linear	{'C': 2000, 'gamma': 'auto', 'kernel': 'linear'}	0.662606	0.713363	0.648881	0.519024	0.714178	0.651610	0.071324	
28	0.084430	0.003175	0.027101	0.000626	2000	scale	rbf	{'C': 2000, 'gamma': 'scale', 'kernel': 'rbf'}	0.771783	0.817873	0.759550	0.705651	0.798682	0.770708	0.038380	
29	0.100305	0.037617	0.008102	0.001194	2000	scale	poly	{'C': 2000, 'gamma': 'scale', 'kernel': 'poly'}	0.795966	0.851588	0.773391	0.692704	0.791832	0.781096	0.051338	
30	0.063302	0.001830	0.010847	0.000295	2000	scale	sigmoid	{'C': 2000, 'gamma': 'scale', 'kernel': 'sigmo...	0.209428	0.461392	0.423484	0.249701	0.371164	0.343034	0.097810	
31	0.193235	0.235915	0.007439	0.000979	2000	scale	linear	{'C': 2000, 'gamma': 'scale', 'kernel': 'linear'}	0.662606	0.713363	0.648881	0.519024	0.714178	0.651610	0.071324	
32	0.116600	0.010807	0.026801	0.001125	3000	auto	rbf	{'C': 3000, 'gamma': 'auto', 'kernel': 'rbf'}	0.798685	0.854436	0.779389	0.724329	0.830400	0.797448	0.044738	
33	0.175601	0.062218	0.007628	0.000514	3000	auto	poly	{'C': 3000, 'gamma': 'auto', 'kernel': 'poly'}	0.805347	0.866836	0.785356	0.722960	0.811140	0.798328	0.046380	
34	0.060174	0.002069	0.011231	0.000607	3000	auto	sigmoid	{'C': 3000, 'gamma': 'auto', 'kernel': 'sigmoid'}	-0.131094	0.105139	0.148123	-0.129002	-0.061992	-0.013765	0.118082	
35	0.218438	0.237974	0.006429	0.000668	3000	auto	linear	{'C': 3000, 'gamma': 'auto', 'kernel': 'linear'}	0.662311	0.712121	0.648424	0.517895	0.714177	0.650986	0.071524	
36	0.125332	0.014649	0.026103	0.000967	3000	scale	rbf	{'C': 3000, 'gamma': 'scale', 'kernel': 'rbf'}	0.798566	0.854354	0.779380	0.724428	0.830494	0.797444	0.044699	
37	0.145431	0.024733	0.007583	0.000217	3000	scale	poly	{'C': 3000, 'gamma': 'scale', 'kernel': 'poly'}	0.805630	0.867141	0.785381	0.722301	0.810836	0.798258	0.046675	
38	0.060398	0.003199	0.011122	0.000611	3000	scale	sigmoid	{'C': 3000, 'gamma': 'scale', 'kernel': 'sigmo...	-0.216531	0.202286	0.146739	-0.171317	-0.089917	-0.025748	0.169386	

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C	param_gamma	param_kernel	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_s
39	0.165314	0.163089	0.005338	0.001999	3000	scale	linear	{'C': 3000, 'gamma': 'scale', 'kernel': 'linear'}	0.662311	0.712121	0.648424	0.517895	0.714177	0.650986	0.071524	

In [ ]:

In [ ]:
```
import pandas as pd
dataset=pd.read_csv("insurance.csv")
dataset
```

In [ ]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

In [ ]:
```
x=dataset[['age', 'bmi', 'children','sex_male', 'smoker_yes','region_northwest','region_southeast','region_southwest']]
y=dataset[['charges']]
print(x)
print(y)
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.20,random_state=42)
```

In [7]:
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

In [13]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeRegressor
param_grid={
    'criterion':['squared_error','absolute_error','poisson'],
    'max_features':['sqrt','log2'],
    'splitter':['best','random']
}
grid=GridSearchCV(DecisionTreeRegressor(),param_grid,refit=True,verbose=3,n_jobs=-1)
grid.fit(x_train,y_train)
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
```

Out[13]:

GridSearchCV

best_estimator_: DecisionTreeRegressor

DecisionTreeRegressor

In [ ]:
```
grid_pred=grid.predict(x_test)
print("prediction data:",grid_pred)
print("actual data:",y_test)
```

In [15]:
```
from sklearn.metrics import r2_score
result=r2_score(grid_pred,y_test)
result
```

Out[15]:

```
0.6436529900287113
```

In [16]:
```
re=grid.cv_results_
print("r_score of best parameter{}:".format(grid.best_params_),result)
```

```
r_score of best parameter{'criterion': 'absolute_error', 'max_features': 'log2', 'splitter': 'best'}: 0.6436529900287113
```

In [17]:
```
table=pd.DataFrame.from_dict(re)
table
```

Out[17]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_features	param_splitter	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_s
0	0.004193	0.000283	0.001982	0.000124	squared_error	sqrt	best	{'criterion': 'squared_error', 'max_features':...	0.661550	0.793567	0.567274	0.583522	0.586958	0.638574	0.084059	
1	0.004756	0.001195	0.003114	0.001834	squared_error	sqrt	random	{'criterion': 'squared_error', 'max_features':...	0.388494	0.676110	0.106813	0.497162	0.544377	0.442591	0.191595	
2	0.005125	0.000628	0.003688	0.002038	squared_error	log2	best	{'criterion': 'squared_error', 'max_features':...	0.624582	0.615947	0.583511	0.644970	0.666035	0.627009	0.027806	
3	0.005484	0.001408	0.002529	0.000458	squared_error	log2	random	{'criterion': 'squared_error', 'max_features':...	0.658601	0.567147	0.665981	0.520670	0.625321	0.607544	0.055708	
4	0.010098	0.002355	0.002058	0.000119	absolute_error	sqrt	best	{'criterion': 'absolute_error', 'max_features'...	0.767554	0.744037	0.242304	0.145316	0.534659	0.486774	0.254430	
5	0.008178	0.000895	0.001917	0.000226	absolute_error	sqrt	random	{'criterion': 'absolute_error', 'max_features'...	0.636885	0.500585	0.361364	0.597929	0.745607	0.568474	0.129944	
6	0.009837	0.000806	0.001926	0.000095	absolute_error	log2	best	{'criterion': 'absolute_error', 'max_features'...	0.758751	0.780678	0.482683	0.667201	0.716696	0.681202	0.106560	
7	0.009104	0.000803	0.001991	0.000168	absolute_error	log2	random	{'criterion': 'absolute_error', 'max_features'...	0.565654	0.620178	0.522871	0.629059	0.575046	0.582561	0.038684	
8	0.003637	0.000240	0.001890	0.000092	poisson	sqrt	best	{'criterion': 'poisson', 'max_features': 'sqrt...	0.554116	0.673933	0.517038	0.606474	0.519676	0.574247	0.059384	
9	0.003201	0.000239	0.001713	0.000056	poisson	sqrt	random	{'criterion': 'poisson', 'max_features': 'sqrt...	0.707482	0.413273	0.352160	0.493286	0.641372	0.521514	0.134271	
10	0.005343	0.001470	0.001922	0.000089	poisson	log2	best	{'criterion': 'poisson', 'max_features': 'log2...	0.631314	0.742153	0.519299	0.631226	0.643013	0.633401	0.070637	
11	0.003741	0.000315	0.002047	0.000263	poisson	log2	random	{'criterion': 'poisson', 'max_features': 'log2...	0.630480	0.693258	0.455184	0.441396	0.520371	0.548138	0.098636	

In [ ]:

In [ ]:
```
import pandas as pd
dataset=pd.read_csv("insurance.csv")
dataset
dataset=pd.get_dummies(dataset,drop_first=True)
print(dataset)
x=dataset[['age','bmi','children','sex_male','smoker_yes','region_northwest','region_southeast','region_southwest']]
y=dataset[['charges']].ravel()
print(x)
print(y)
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.20,random_state=42)
```

In [ ]:
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

In [24]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor
param_grid={
    "criterion":['squared_error','absolute_error','poisson'],
    'max_features':['sqrt','log2'],
    'n_estimators':[10,100]
}
grid=GridSearchCV(RandomForestRegressor(),param_grid,refit=True,verbose=3,n_jobs=-1)
grid.fit(x_train, y_train)
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
```

```
C:\Users\User\anaconda3\Lib\site-packages\sklearn\base.py:1403: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().
  return fit_method(estimator, *args, **kwargs)
```

Out[24]:

▸ GridSearchCV
▸ best_estimator_: RandomForestRegressor
▸ RandomForestRegressor

In [ ]:
```
grid_pred=grid.predict(x_test)
print("prediction data:",grid_pred)
print("actual data:",y_test)
```

In [26]:
```
from sklearn.metrics import r2_score
result=r2_score(grid_pred,y_test)
result
```

Out[26]:

```
0.10014519146478629
```

In [27]:
```
re=grid.cv_results_
print("r_score of best parameter{}:".format(grid.best_params_),result)
```

```
r_score of best parameter{'criterion': 'poisson', 'max_features': 'log2', 'n_estimators': 100}: 0.10014519146478629
```

In [28]:
```
table=pd.DataFrame.from_dict(re)
table
```

Out[28]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_features	param_n_estimators	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_s
0	0.049928	0.012107	0.008556	0.003646	squared_error	sqrt	10	{'criterion': 'squared_error', 'max_features':...	0.796618	0.836715	0.770167	0.747612	0.798151	0.789853	0.029941	
1	0.520552	0.102802	0.026899	0.009053	squared_error	sqrt	100	{'criterion': 'squared_error', 'max_features':...	0.826991	0.891856	0.801031	0.770358	0.826508	0.823349	0.040056	
2	0.046281	0.019029	0.009313	0.005671	squared_error	log2	10	{'criterion': 'squared_error', 'max_features':...	0.819946	0.877237	0.770064	0.748872	0.815264	0.806276	0.044519	
3	0.331510	0.005878	0.024573	0.002425	squared_error	log2	100	{'criterion': 'squared_error', 'max_features':...	0.827008	0.899964	0.810339	0.781926	0.841312	0.832110	0.039261	
4	0.069250	0.009209	0.005595	0.002600	absolute_error	sqrt	10	{'criterion': 'absolute_error', 'max_features'...	0.811293	0.868661	0.776291	0.740022	0.773544	0.793962	0.043632	
5	0.616470	0.012877	0.021819	0.002103	absolute_error	sqrt	100	{'criterion': 'absolute_error', 'max_features'...	0.826681	0.891308	0.796395	0.776353	0.824147	0.822977	0.038896	
6	0.071761	0.002187	0.004397	0.000551	absolute_error	log2	10	{'criterion': 'absolute_error', 'max_features'...	0.810515	0.881049	0.762347	0.777911	0.789132	0.804191	0.041510	
7	0.650082	0.059575	0.021372	0.002372	absolute_error	log2	100	{'criterion': 'absolute_error', 'max_features'...	0.828252	0.901697	0.809840	0.782973	0.842594	0.833071	0.039681	
8	0.029434	0.000804	0.003910	0.000253	poisson	sqrt	10	{'criterion': 'poisson', 'max_features': 'sqrt...	0.786889	0.868971	0.769616	0.752938	0.784938	0.792670	0.040063	
9	0.251572	0.001734	0.019498	0.000487	poisson	sqrt	100	{'criterion': 'poisson', 'max_features': 'sqrt...	0.825863	0.886140	0.794821	0.768311	0.821871	0.819401	0.039284	
10	0.029455	0.000814	0.003959	0.000294	poisson	log2	10	{'criterion': 'poisson', 'max_features': 'log2...	0.829576	0.869373	0.781763	0.766995	0.824411	0.814424	0.036510	
11	0.250871	0.022722	0.016671	0.003547	poisson	log2	100	{'criterion': 'poisson', 'max_features': 'log2...	0.828228	0.903952	0.813634	0.779748	0.845052	0.834123	0.041004	

In [ ]: